Guía completa de JSDoc para JavaScript

Tabla de contenido

Actualiza la tabla de contenido en Word (clic derecho > Actualizar campo) si no aparece automáticamente.

1. Introducción

JSDoc es un estándar de comentarios con anotaciones estructuradas para describir el código JavaScript. Permite documentar funciones, clases, módulos, tipos y contratos de datos mediante etiquetas específicas. Sus beneficios incluyen autocompletado y validación de tipos en editores como VS Code, generación de documentación HTML y mejor comunicación dentro de equipos.

1.1 Beneficios clave

- Mejor autocompletado y tooltips en el editor (intellisense).
- Chequeo de tipos en JavaScript con TypeScript (// @ts-check o checkJs).
- Documentación navegable (HTML/PDF) generada automáticamente.
- Menos errores por contratos de API ambiguos.

2. Instalación y uso básico

Instala JSDoc como dependencia de desarrollo y genera documentación a partir de tus comentarios:

```
# Instalación
npm i -D jsdoc

# Generación básica (escanea ./src y deja la salida en ./docs)
npx jsdoc -r src -d docs
```

Estructura básica de un comentario JSDoc:

```
/**
 * Suma dos números.
 * @param {number} a - Primer sumando.
 * @param {number} b - Segundo sumando.
 * @returns {number} La suma de a y b.
 */
function sumar(a, b) {
  return a + b;
}
```

IES MARIA ENRIQUEZ

3. Documentar distintas entidades

3.1 Funciones

Usa @param y @returns. Para funciones asíncronas, documenta el tipo Promise<T>. Para generadores, usa @yields.

```
/**
 * Obtiene un usuario remoto.
 * @param {string} id - Identificador del usuario.
 * @returns {Promise<{id: string, nombre: string}>} Usuario encontrado.
 * @throws {Error} Si la petición falla.
 */
async function getUser(id) { /* ... */ }

/**
 * Genera números del 0 al n-1.
 * @param {number} n
 * @yields {number}
 */
function* range(n) { for (let i=0; i<n; i++) yield i; }
```

3.2 Objetos, tipos y propiedades

Declara tipos de objetos con @typedef y sus propiedades con @property.

```
/**
 * @typedef {Object} Usuario
 * @property {string} id
 * @property {string} nombre
 * @property {number} [edad] - Opcional
 * @property {"admin"|"editor"|"viewer"} rol
 */

/** @type {Usuario} */
const u1 = { id: '42', nombre: 'Ada', rol: 'admin' };
```

3.3 Clases y constructores

Anota clases, sus miembros y herencia con @class, @constructor, @extends, @implements.

```
/**
 * Representa un punto en 2D.
 * @class
 */
class Punto {
  /** @param {number} x @param {number} y */
  constructor(x, y) { this.x = x; this.y = y; }
  /** @returns {number} */
  distanciaOrigen() { return Math.hypot(this.x, this.y); }
}

/**
 * @extends Punto
 */
class PuntoColor extends Punto {
  /** @param {number} x @param {number} y @param {string} color */
```

IES MARIA ENRIQUEZ

```
  constructor(x, y, color) { super(x,y); this.color = color; }
}
```

3.4 Módulos y exports

Documenta módulos CommonJS o ES con @module y @exports/@alias/@name cuando sea necesario.

```
/** @module calculadora */

/** Suma dos números. */
export function add(a, b) { return a + b; }

/** Resta dos números. */
export function sub(a, b) { return a - b; }
```

3.5 Tipos avanzados y utilidades

- Uniones: {string|number}
- Opcionales: [nombre], o {type=} con valor por defecto documentado con @default
- Nullable: {?number} (puede ser null) y no-null: {!Object}
- Arreglos: {Array<string>} o {string[]} (TS),
- Funciones: {function(string, number): boolean}
- Genéricos: @template T luego usar {T}, Promise<T>, Array<T>
- Tipos importados (TS): {import("./tipos").Usuario}

```
/**
 * Filtra una lista con un predicado genérico.
 * @template T
 * @param {T[]} lista
 * @param {(x: T) => boolean} pred
 * @returns {T[]}
 */
function filtrar(lista, pred) { return lista.filter(pred); }
```

4. Etiquetas (tags) más comunes

Listado de etiquetas útiles en proyectos modernos:

- @param {Tipo} nombre - describe el parámetro
- @returns {Tipo} - valor devuelto
- @type {Tipo} - anota una variable/constante/propiedad
- @typedef Nombre - define un alias de tipo
- @property {Tipo} nombre - propiedad dentro de un typedef
- @template T,U - tipos genéricos
- @example - bloque de ejemplo(s) de uso
- @see y {@link URL|Texto} - referencias
- @deprecated - marca APIs en desuso
- @private/@protected/@public/@package - visibilidad
- @readonly - solo lectura
- @enum {Tipo} - valores enumerados
- @yields {Tipo} - para generadores

IES MARIA ENRIQUEZ

- @throws {ErrorTipo} - errores esperados
- @default - valor por defecto
- @summary y @description - resúmenes y descripciones largas
- @fileoverview - descripción general de archivo
- @version y @since - versionado y fecha de introducción
- @this {Tipo} - contexto this explícito

```
/**
 * @fileoverview Utilidades de arrays.
 * @author Tu Nombre
 * @version 1.0.0
 * @since 2025-01-01
 */

/**
 * @deprecated Usa `nuevaApi()` en su lugar.
 * @see {@link https://ejemplo.com/migracion|Guía de migración}
 */
export function viejaApi() {}
```

5. Chequeo de tipos en JS con TypeScript

Puedes activar comprobación de tipos en archivos .js usando JSDoc y TypeScript:

```
// En el tope de tus archivos .js
// @ts-check

// O a nivel de proyecto (jsconfig.json/tsconfig.json)
{
  "compilerOptions": {
    "checkJs": true,
    "strict": true
  },
  "include": ["src/**/*.js"]
}

// Importa tipos TS en JSDoc
/** @type {import('./tipos').Usuario} */
const usuario = { id: '1', nombre: 'Lin' };
```

Esto habilita errores de tipo en el editor y en CI si compilas con tsc --noEmit, sin migrar aún a TypeScript completo.

6. Integración con editores y linters

- VS Code: habilita "checkJs" y disfruta de intellisense basado en JSDoc.
- ESLint: usa eslint-plugin-jsdoc para reglas sobre formato y completitud de comentarios.
- Prettier: mantén consistencia de formato; evita romper bloques JSDoc.

IES MARIA ENRIQUEZ

7. Generación de documentación HTML con JSDoc

Configura JSDoc con un archivo jsdoc.json, apunta a tus fuentes y elige carpeta de salida:

```
// jsdoc.json
{
  "source": { "include": ["src"], "includePattern": ".+\\.js(doc|x)?$" },
  "plugins": [],
  "opts": { "destination": "docs", "recurse": true, "template": "default" }
}

# Ejecuta
npx jsdoc -c jsdoc.json
```

Plantillas: la plantilla "default" viene con JSDoc. Hay temas de la comunidad (minami, docdash, etc.). Verifica su mantenimiento antes de adoptarlos.

8. Casos prácticos

8.1 API de librería con módulos ES

```
/** @module color */

/**
 * Convierte RGB a hex.
 * @param {{r:number,g:number,b:number}} rgb
 * @returns {`#${string}`}
 * @example
 * rgbToHex({r:255,g:0,b:0}) // => "#ff0000"
 */
export function rgbToHex({r,g,b}) {
  return '#' + [r,g,b].map(x => x.toString(16).padStart(2,'0')).join('');
}
```

8.2 Modelado de datos con @typedef

```
/**
 * @typedef Order
 * @property {string} id
 * @property {"created"|"paid"|"shipped"|"cancelled"} status
 * @property {{sku: string, qty: number}[]} items
 * @property {{street: string, city: string, country: string}} shipping
 */

/**
 * @param {Order} order
 * @returns {boolean}
 */
export function isShippable(order) { return order.status === 'paid'; }
```

9. Buenas prácticas

- Escribe @param/@returns para TODA función pública.
- Prefiere tipos específicos a any/object.
- Usa ejemplos reales en @example y mantenlos actualizados.

IES MARIA ENRIQUEZ

- Enlaza tipos y recursos con {@link}.
- Mantén consistencia en idioma, tono y formato.
- Automatiza: lint de JSDoc en CI, generación de docs en releases.

10. Apéndice: referencia rápida de etiquetas

Etiqueta	Propósito	Ejemplo
@param	Parámetro de función	@param {number} a - primer sumando
@returns	Valor de retorno	@returns {Promise<User>}
@type	Anotar variable/propiedad	/** @type {HTMLDivElement} */
@typedef	Alias de tipo	/** @typedef {Object} User */
@property	Propiedad dentro de typedef	@property {string} id
@template	Genéricos	@template T
@example	Ejemplo de uso	@example doThing(42)
@see/@link	Referencia externa	@see {@link https://...}
@deprecated	API en desuso	@deprecated usar nueva()
@private	Visibilidad privada	@private
@protected	Visibilidad protegida	@protected
@public	Visibilidad pública	@public
@package	Visibilidad de paquete	@package
@readonly	Solo lectura	@readonly
@enum	Enumeración	@enum {"up"|"down"}
@yields	Yield de generador	@yields {number}
@throws	Errores esperados	@throws {TypeError}
@default	Valor por defecto	@default 10
@summary	Resumen breve	@summary Cálculo de totales
@description	Descripción larga	@description Explica detalles...
@fileoverview	Descripción de archivo	@fileoverview helpers
@version/@since	Versionado	@since 2025-01-01
@this	Contexto this	@this {MyCtx}

IES MARIA ENRIQUEZ